Week 8 (Days 50-56): Dimensionality, anomalies, recommenders, time series, calibration, ML strategy, and Rust

Status: written in full. Titles match `SYLLABUS.md`.

Week goals

1. Compress data with PCA (built from the SVD), and visualise it with t-SNE, knowing what each view preserves.
2. Detect anomalies with density models and isolation forests, with thresholds tuned on validation data.
3. Build a recommender from biases plus matrix factorisation on MovieLens 100K.
4. Forecast with lag features without leaking the future.
5. Check whether probabilities mean what they say, and explain effects with partial dependence.
6. Diagnose a model systematically (learning curves, error analysis) and decide what to try next.
7. Write a Rust/PyO3 k-means kernel and benchmark it honestly against NumPy and scikit-learn.

```
flowchart LR
  PCA[PCA / t-SNE] --> AN[Anomaly detection]
  AN --> RS[Recommenders: MF]
  RS --> TS[Time series: lags]
  TS --> CAL[Calibration & PDP]
  CAL --> ST[ML strategy: learning curves]
  ST --> RU[Project: Rust k-means kernel]
```

New dataset: MovieLens 100K (GroupLens Research: 100,000 ratings from 943 users on 1,682 films). We use its official `u1.base`/`u1.test` split (80k/20k). Download URL is in `data/README.md`; the data comes with its own usage licence (read the README inside the zip).

Day 50: PCA in practice; t-SNE/UMAP for visualisation

Review today: Day 49 (+1), Day 47 (+3), Day 43 (+7), Day 29 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. Picture a cloud of points shaped like a flattened rugby ball in 3-D. Most of the spread runs along its long axis, some along the second, and almost none along the third. **PCA** finds those axes, the directions of maximum variance, each orthogonal to the previous ones. Keep the first k and you have a compressed description that loses as little variance as possible.

Math (connecting to Day 11). Centre the data, Xc = X - mean. The SVD Xc = USumV^T gives: - the principal directions as the rows of V^T, - the projected coordinates (scores) Xc.V?, - the variance along component j, sigma?^2/(m - 1).

The **explained variance ratio** is the share of total variance each component carries. Reconstruction from k components, X? = mean + Xc V?V?^T, has mean squared error equal to the sum of the *dropped* eigenvalues divided by the number of features. That is a check you can run.

Uses. 1. Visualisation (2-D or 3-D). 2. Denoising and compression. 3. Speed-ups before slow models. 4. Decorrelating features.

PCA is linear and variance-driven, so scale features first (unless they share units, like pixels). High variance does not have to mean useful for prediction.

t-SNE and UMAP are *non-linear* maps that try to keep each point's near neighbours near. They make beautiful cluster pictures, but distances between clusters, cluster sizes and axes carry no meaning. They are stochastic, and they are for looking, not for features. **Trustworthiness** (in [0, 1]) measures how many of a point's neighbours in the 2-D map are true high-dimensional neighbours. UMAP is a separate package that is not installed in this course's environment. The concepts carry over.

Block 2: Worked example (45 min)

`code/week-08/day50_pca.py` on digits (1797 x 64 pixels). Verified: - Scratch SVD PCA matches scikit-learn. The first 5 explained-variance ratios are [0.1489, 0.1362, 0.1179, 0.0841, 0.0578], and the components agree up to sign. (Sign is arbitrary: v and -v are the same axis.) - Components needed: 50% of variance -> 5, 80% -> 13, 90% -> 21, 95% -> 29, 99% -> 41 (of 64). - Reconstruction MSE per pixel: k=2 13.421, k=10 4.914, k=20 1.984, k=40 0.221. Each **exactly** equals (sum of dropped eigenvalues)/64. - PCA(k) + logistic regression (5-fold): k=5 0.7713, k=10 0.8403, k=20 0.8993, all 64 0.9199. Compression costs accuracy here, because digits aren't high-dimensional enough to need it. - t-SNE took about 2 s. Trustworthiness at k=10: **t-SNE 0.9927** vs PCA-2D 0.8300. t-SNE preserves local neighbourhoods far better. Open `out/day50_pca_tsne.png` and compare how well the digit groups separate in each view.

Block 3: Coding lab (2h)

1. Implement `pca_fit(X, k)` returning the mean, components and explained variance via `np.linalg.svd(full_matrices=False)`. Then implement `transform` and `inverse_transform`. Verify the reconstruction identity.
2. Show reconstructions of 5 digits at k = 2, 5, 10 and 20 as 8x8 images.
3. Plot the cumulative explained variance and mark the 90% point.
4. Run t-SNE with perplexity 5, 30 and 100. How do the pictures differ? Would you draw conclusions about *between*-cluster distance from any of them?
5. Stretch: on California (8 features), compare PCA on scaled vs unscaled data. Which feature dominates PC1 when unscaled, and why?

Block 4: Practice problems (1h)

1. Eigenvalues 5, 3, 1, 1 (total 10). Explained ratio of PC1 and PC2 together? *(0.8.)*
2. Why centre before PCA? *(Otherwise PC1 points at the mean, not along the spread.)*
3. Is the sign of a principal component meaningful? *(No.)*
4. What does trustworthiness 0.99 vs 0.83 say? *(Neighbours in the t-SNE map are mostly true neighbours, far more than in the 2-D PCA view.)*
5. Can you feed t-SNE coordinates to a classifier for new points? *(Not directly; standard t-SNE has no transform for new data.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- How do you get PCA from the SVD?
- Define explained variance ratio.
- What equals the reconstruction error?
- Name three things t-SNE plots do *not* preserve.
- How many components gave 90% on digits?

Going deeper (optional)

Probabilistic PCA treats x = Wz + mu + noise with a Gaussian z, which links PCA to factor analysis and, later, to autoencoders. A linear autoencoder with squared loss learns the same subspace as PCA (not necessarily orthogonal axes). Month 3 will build the non-linear version.

Common mistakes / how to debug

- Fitting PCA on all data before CV. It is a fitted transform, so put it in the pipeline.
- Reading t-SNE cluster sizes as real.

What to do next

Tomorrow: modelling "normal" so the abnormal stands out.

Day 51: Anomaly detection

Review today: Day 50 (+1), Day 48 (+3), Day 44 (+7), Day 30 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. Fraud, machine faults and intrusions are rare, varied and often new. You may have few or no labelled examples. Instead of learning "what anomalies look like", learn **what normal looks like**, and flag points that normal data would rarely produce.

The density approach (Ng-style). 1. Fit a density p(x) on normal training data. 2. Flag x when p(x) < eps. 3. Choose eps on a small **labelled validation set** for best F1.

- **Independent Gaussians:** p(x) = ?? N(x?; mu?, sigma?^2). Cheap, but blind to correlations. A point can be normal on each feature separately and still be an anomaly jointly.
- **Multivariate Gaussian:** p(x) = N(x; mu, Sum). It captures correlations, needs m ? n, and Sum must be invertible (add a small ridge).
- Work in **log p**. Transform skewed features first (log), so that a Gaussian fits.

Isolation Forest. Build random trees that split on random features at random thresholds. Anomalies are few and different, so they get isolated in **fewer splits**. The anomaly score comes from the average path length. It needs no density assumption and scales well.

Anomaly detection vs supervised learning. With plenty of labelled anomalies of *stable* types, a supervised classifier is better. Use anomaly detection when anomalies are rare, diverse or new.

Block 2: Worked example (45 min)

`code/week-08/day51_anomaly.py` uses breast cancer as a stand-in. Benign = "normal" and malignant = "anomaly", with anomalies made rare. Training has **237 normal rows only**. Validation has 80 rows (20 anomalies), and so does test. Features are log-transformed and scaled. Verified:

method	test precision	recall	F1	AP
independent Gaussians	0.765	0.650	0.703	0.831
multivariate Gaussian	**0.895**	**0.850**	**0.872**	**0.950**
Isolation Forest	0.867	0.650	0.743	0.878

The multivariate Gaussian wins because the 30 features are strongly correlated (radius, perimeter and area), and only it models that. Its chosen threshold, log p = -2.0e8, looks absurd. It reflects a nearly singular covariance (some directions have tiny variance), so anomalies that move along those directions get astronomically small densities. The *ranking* is still excellent. Treat a raw log-density value as a score, not a meaningful probability.

Block 3: Coding lab (2h)

1. Implement both Gaussian detectors with `scipy.stats.norm.logpdf` and `multivariate_normal.logpdf`, plus `tune_epsilon(scores_val, y_val)`.
2. Change the covariance ridge from 1e-3 to 1e-1 and 1. How do the threshold and F1 change?
3. Add the Mahalanobis distance (x - mu)^TSum?^1(x - mu) as a score. Show that it ranks identically to the multivariate log p.
4. Isolation Forest: sweep `n_estimators` and `max_samples`, and plot the score histograms for normal vs anomaly.
5. Stretch: inject 5% anomalies into the *training* set (contamination) and re-measure all three. Which one is most robust?

Block 4: Practice problems (1h)

1. Features x? ~ N(0, 1) and x? ~ N(0, 1), with correlation 0.99. Is x = (2, -2) anomalous under the independent model? Under the multivariate model? *(Mildly under independent; extremely under multivariate.)*
2. Why choose eps on validation data with labels? *(The density alone doesn't say where "rare enough" lies.)*
3. Why log-transform skewed features first? *(Gaussian fit.)*
4. Isolation Forest's intuition in one line? *(Anomalies get isolated in fewer random splits.)*
5. When would you prefer a supervised classifier? *(Many labelled anomalies of stable types.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.

- **Self-check:**
- Describe the density-threshold recipe.
- What does the multivariate Gaussian capture that independent Gaussians miss?
- What is Isolation Forest's score based on?
- Which method won here, and why?
- Why is a log p of -2e8 not alarming?

Going deeper (optional)

For high-dimensional data (images, logs), density estimation is hard. Common substitutes are **reconstruction error** (PCA or autoencoder: normal data reconstructs well) and **one-class SVM**. Try PCA reconstruction error with k = 5 here and compare its AP.

Interview-style question (15 min, written)

"We have 10 labelled frauds and 100,000 normal transactions. Supervised or anomaly detection?" Write your answer. With 10 positives a supervised model cannot learn the variety of fraud, and its validation estimates would be noise. Fit a density or isolation model on normal data, and use the 10 frauds only to choose the threshold and to sanity-check the ranking (average precision). Plan to switch to, or combine with, a supervised model as labelled fraud accumulates. Mention the operating cost: every flagged transaction costs analyst time, so report precision at the review capacity (for example, the top 100 scores per day), not only F1.

Common mistakes / how to debug

- Tuning eps on the test set.
- Fitting the density on data that contains anomalies, without knowing it.

What to do next

Tomorrow: recommendation, which means predicting a user's rating of a film they haven't seen.

Day 52: Recommender systems: collaborative filtering, matrix factorisation

Review today: Day 51 (+1), Day 49 (+3), Day 45 (+7), Day 31 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The problem. A huge, mostly empty matrix R of users x items, with ratings where known. Predict the blanks and recommend the highest. MovieLens 100K is about 5% filled.

Content-based vs collaborative. - **Content-based** recommends items similar to what you liked, using item features (genres). - **Collaborative filtering** uses only the rating pattern: people who agreed with you before will agree again.

Baseline predictors first. r??_i = mu + b? + b_i. Here mu is the global mean, b? says "this user rates generously" and b_i says "this film is loved". Fit them with regularisation lambda, because a film with 1 rating shouldn't get a big bias. This baseline is surprisingly strong.

Matrix factorisation (MF). Give each user a vector p? in ?? and each item q_i in ?? (latent "taste" factors), so r??_i = mu + b? + b_i + p?.q_i. Minimise the squared error **on observed entries only**, plus lambda(||p||^2 + ||q||^2 + b^2). Use **SGD**: for each rating, e = r - r?, then p? += ?(e.q_i - lambdap?), and similarly for q_i and the biases. Unlike SVD on a filled matrix, this ignores missing entries rather than treating them as zeros.

Evaluation. RMSE on held-out ratings, plus ranking metrics (precision@k), and remember the **cold start** problem: new users and items have no ratings.

Block 2: Worked example (45 min)

`code/week-08/day52_recsys.py` on the official u1 split (80,000 train / 20,000 test ratings; 943 users, 1,650 items in train; density 0.0504). Verified test RMSE:

model	test RMSE
global mean (3.5284)	1.1537
user + item biases, lambda=0	0.9564
user + item biases, lambda=5	**0.9550**
user + item biases, lambda=25	0.9718
MF k=20, lambda=0, epoch 10 -> 30	0.9524 -> **1.0173**
MF k=20, lambda=0.05, epoch 10 -> 20 -> 30	0.9529 -> **0.9348** -> 0.9400

Lessons: 1. Biases alone take you from 1.154 to 0.955. Most of the achievable gain is "who is generous, which films are good". 2. **Unregularised MF overfits badly:** of the printed checkpoints it is best at epoch 10, and by epoch 30 it is worse than the bias baseline. 3. With lambda = 0.05, MF reaches **0.9348** at epoch 20, beating the baseline by 0.02. That is modest but real, and it is the "taste" signal. 4. Even regularised MF starts drifting upwards by epoch 30, so early-stop on validation data.

Pure-Python SGD took about 18 s per configuration (30 epochs x 80k updates). Lab step 5 vectorises it.

The bias fit printed a divide warning for items with no training ratings (0/0). The script now suppresses it, and those items fall back to bias 0. That is the cold-start problem in miniature.

Block 3: Coding lab (2h)

1. Fit biases by alternating closed-form updates (b_i = Sum(r - mu - b?)/(n_i + lambda), then the same for b?), and reproduce the lambda table.
2. Implement MF-SGD. Add a validation split carved from u1.base, plus early stopping.
3. Sweep k in {5, 20, 50} and lambda in {0.02, 0.05, 0.1}, and record the best epoch and RMSE.
4. For one user, list the top 10 unseen films by predicted rating, with titles from `u.item`. Are they plausible? Are they just globally popular?
5. Stretch (performance): rewrite one SGD epoch with Numba, or as a Rust/PyO3 function in the style of Day 56. Report the speed-up honestly.

Block 4: Practice problems (1h)

1. mu = 3.5, b? = 0.3, b_i = -0.5, p?.q_i = 0.4. Prediction? *(3.7.)*
2. Why train only on observed entries? *(Missing != zero; filling biases everything towards "dislike".)*
3. Why regularise biases? *(Items with few ratings would get extreme biases.)*
4. A new film with zero ratings. What does MF predict? *(mu plus the user bias; cold start needs content features.)*
5. RMSE 0.93 vs 0.95. Does that matter to users? *(Maybe not. Check ranking metrics and A/B tests.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Write the biased MF prediction.
- Write the SGD update for p?.
- How much did biases improve on the global mean?
- What happened to unregularised MF?
- Define cold start.

Going deeper (optional)

Implicit feedback (clicks and views, not stars) needs different losses. Positive-only data uses weighted MF or pairwise ranking losses (BPR: "the clicked item should score above a random unclicked one"). Modern systems use two-tower neural models. Their dot-product scoring is exactly MF's p?.q_i, with the vectors produced by networks from features.

Common mistakes / how to debug

- Random splits that leak a user's future ratings into training (u1 is a random split; time-based splits are stricter).
- Predictions outside [1, 5]. Clip them.

What to do next

Tomorrow: time series, where time order is everything.

Day 53: Time-series basics: lag features, leakage-safe validation

Review today: Day 52 (+1), Day 50 (+3), Day 46 (+7), Day 32 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Intuition. Tomorrow's bike demand depends on today's. Time series have **autocorrelation** (values near in time are similar), **seasonality** (daily and weekly cycles) and **trend**. This breaks the usual assumption that rows are independent. A random split puts the "future" in training, and its neighbours in validation become near-copies.

Supervised framing. Predict y? from features available *at prediction time*: - calendar (hour, weekday), - exogenous inputs known in advance (the weather forecast), - **lags** y???, y???? (same hour yesterday), y????? (same hour last week), - **rolling statistics of the past**, such as the mean of y????...y???.

Never use a window that includes t or later.

Validation. `TimeSeriesSplit` trains on an expanding past and tests on the next block. Always compare with **naive baselines**: "same as an hour ago" and "same as last week". If you can't beat them, your model isn't learning anything useful.

The forecast horizon decides your features. Predicting 24 hours ahead means lag 1 is not available. Use lags >= 24.

Block 2: Worked example (45 min)

`code/week-08/day53_timeseries.py` on hourly bike demand (HistGB regressor, `TimeSeriesSplit(5)` with 1500-hour test blocks, vs shuffled `KFold(5)`). Verified MAE (rentals per hour):

features	TimeSeriesSplit	shuffled KFold
naive "same as 1 hour ago"	65.3	-
naive "same hour last week"	64.7	-
calendar + weather only	48.7	26.5
+ lags (1, 2, 24, 168) and past 24-h mean	**33.8**	22.3
+ LEAKY centred rolling mean (includes t+1)	18.3	11.7

Three lessons: 1. **Shuffled CV is wildly optimistic:** 26.5 vs 48.7 for the same model. Random folds let the model interpolate between neighbouring hours it already saw. 2. **Lags genuinely help** under honest validation (48.7 -> 33.8), and clearly beat both naive baselines. 3. **The leaky feature looks like a breakthrough** (18.3), but it uses y???, which you will never have at prediction time. A sudden big improvement from one feature is a red flag. Check its timestamp.

The fold boundaries are printed (for example train up to row 9709 -> test 9710-11209). Read them to confirm that the test always lies after the training data.

Block 3: Coding lab (2h)

1. Build lag and rolling features with `shift` and `rolling` (always `shift(1)` *before* `rolling`). Assert there are no NaNs after the warm-up rows are dropped.
2. Reproduce the table. Then build a **24-hour-ahead** feature set (only lags >= 24) and report its honest MAE.
3. Add a `gap=24` to `TimeSeriesSplit` and explain when a gap is needed.
4. Plot one test week: actual demand vs prediction vs the naive baselines.
5. Stretch: fit a simple seasonal model (hour x weekday means from training only) as a third baseline. How close does it get to HistGB?

Block 4: Practice problems (1h)

1. Which feature leaks: `y.rolling(24).mean()` or `y.shift(1).rolling(24).mean()`? *(The first includes y?.)*
2. Why can't you use lag 1 for a 6-hours-ahead forecast? *(It isn't known 6 hours in advance.)*

3. What does an expanding window mean in TimeSeriesSplit? *(Each fold's training set includes all earlier data.)*
4. Shuffled CV MAE 22 vs time-split MAE 34. Which do you report? *(34.)*
5. Why compare with "same hour last week"? *(It captures seasonality at no cost; your model must beat it.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Define autocorrelation and seasonality.
- What makes a feature legal at prediction time?
- What gap did shuffled CV create?
- What did the leaky feature score?
- How does the forecast horizon constrain the lags?

Going deeper (optional)

Classical models (ARIMA, exponential smoothing) model the series directly and give prediction intervals. Gradient boosting with lags is a strong practical default for many related series. Sequence models (Month 3's RNNs and transformers) learn the lags themselves. In every case the validation discipline stays the same.

Interview-style question (15 min, written)

"Our demand forecast had MAE 22 in testing but 34 live. Why?" Write a short answer referring to today's table. The gap matches shuffled CV against time-ordered evaluation: shuffled folds let the model interpolate between neighbouring hours. Propose three checks: 1. Re-run the validation with `TimeSeriesSplit` and the production horizon. 2. Audit every rolling and lag feature for `center=True` or a missing `shift`. 3. Confirm that the exogenous inputs used in training (observed weather) match what is available live (forecast weather).

State what you would report to stakeholders from now on: the time-split number and the naive baselines beside it.

Common mistakes / how to debug

- Using `rolling(..., center=True)`.
- Normalising with statistics from the whole series.
- Forgetting that holidays shift weekly patterns.

What to do next

Tomorrow: are the model's probabilities honest, and how does a feature move the prediction?

Day 54: Interpretability and probability calibration

Review today: Day 53 (+1), Day 51 (+3), Day 47 (+7), Day 33 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

Calibration. A model is calibrated if, among cases where it says 0.8, about 80% are positive. Ranking (AUC) and calibration are different properties: a model can rank perfectly and still say 0.99 for everything positive-ish. Calibration matters whenever the probability *itself* is used, in expected costs, risk scores, or thresholds chosen by formula (Day 45's c_FP/(c_FP + c_FN)).

Measuring it. - **Reliability diagram:** bin the predictions, then plot mean predicted vs observed frequency. The diagonal is perfect. - **Brier score:** mean (p - y)^2, where lower is better. It mixes calibration and sharpness. - **Log loss:** heavily punishes confident mistakes.

Fixing it (on held-out data, via `CalibratedClassifierCV`): - **Platt / sigmoid scaling:** fit a logistic regression on the model's scores. It has 2 parameters, so it suits small data and S-shaped distortions. - **Isotonic regression:** a monotone step function. It is more flexible and needs more data.

Typical behaviour. Logistic regression is usually well calibrated, because its loss is log loss. Naive Bayes is overconfident (double-counted evidence). SVM margins are not probabilities at all. Boosted trees are usually close.

Interpretability: partial dependence (PDP). For feature j, set every row's x? to a value v, average the predictions, and repeat over a grid of v. The curve shows the model's *average* response to x?. Caveats: it averages away interactions (ICE curves show each row), and with correlated features it creates unrealistic rows.

Block 2: Worked example (45 min)

`code/week-08/day54_calibration.py` uses a synthetic task (30,000 rows, 20 features with 4 informative and 10 redundant; 70/30 split). Verified on test:

model	Brier	log loss	ROC-AUC
logistic regression	0.0777	0.2612	0.9572
Gaussian NB	0.1036	0.5021	0.9403
LinearSVC (sigmoid of raw margin)	0.1142	0.3905	0.9572
Gaussian NB + isotonic	0.0929	0.3106	0.9402
LinearSVC + Platt	**0.0777**	**0.2611**	0.9572

How to read it: 1. LinearSVC and logistic regression have **identical AUC** (0.9572), but the SVM's raw sigmoid-of-margin is badly calibrated (Brier 0.114). After Platt scaling it matches logistic regression exactly. Ranking was never the problem. 2. NB's log loss of 0.50 shows overconfidence. Isotonic calibration cuts it to 0.31. AUC is unchanged, because calibration is monotone. 3. Open `out/day54_reliability.png` and look at the NB curve's S-shape.

PDP for HistGB on California, at income grid points: MedInc 1.6 -> -0.693, 3.23 -> -0.21, 4.86 -> +0.257, 6.49 -> +1.067, 7.3 -> +1.261 (centred, in $100k). That is a strong, roughly monotone income effect that flattens at the top. The 5.0 price cap from Day 35 may contribute to the flattening. Test that by dropping the capped rows.

Block 3: Coding lab (2h)

1. Write `reliability(y, p, bins)` yourself (equal-frequency bins) and compare with `calibration_curve(strategy="quantile")`.
2. Reproduce the table. Then calibrate a random forest on the same data and report Brier before and after.
3. Show that calibration doesn't change AUC (a monotone map), and explain why Brier still improves.
4. Draw the PDP and ICE curves (`PartialDependenceDisplay`, `kind="both"`) for MedInc and AveOccup. Where do the ICE lines fan out?
5. Stretch: compute the expected calibration error (ECE, the weighted mean |observed - predicted| over bins) for every model.

Block 4: Practice problems (1h)

1. 100 cases predicted 0.9 and 60 are positive. Calibrated? *(No, overconfident.)*
2. Brier score for always predicting the base rate 0.3, when the base rate is 0.3? *(0.3.0.7 = 0.21.)*
3. Why calibrate on held-out data? *(On training data the model looks more confident and correct than it is.)*
4. Platt or isotonic for 300 validation rows? *(Platt.)*
5. A PDP is flat but the feature has high permutation importance. Possible why? *(Interactions that cancel on average. Check ICE.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Define calibration.
- Name two calibration methods and when to use each.
- Why did the SVM's AUC equal logistic regression's?
- What does a PDP average over?
- Which model was most overconfident?

Going deeper (optional)

Proper scoring rules (log loss, Brier) are minimised in expectation by the true probabilities, so they reward honesty. Accuracy and F1 are not proper. That is why we train on log loss and *threshold* separately. Deep networks (Month 3) are often overconfident. Temperature scaling, a one-parameter Platt variant on logits, is the standard fix.

Common mistakes / how to debug

- Calibrating and evaluating on the same data.
- Reading PDPs as causal effects.

What to do next

Tomorrow: the most practical day of the month, a systematic way to decide what to try next.

Day 55: Systematic ML strategy: learning curves, error analysis, what to try next

Review today: Day 54 (+1), Day 52 (+3), Day 48 (+7), Day 34 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson (1h15)

The problem. Your model is at 87% and you have two weeks. Options include more data, more features, a bigger model, more regularisation, a different algorithm, better labels and hyperparameter search. Trying them at random wastes months. Ng's practical teaching here is to **diagnose first, then act**. In outline:

1. **Set a single number to optimise** (plus satisficing constraints such as latency < 50 ms), and a dev set that represents the data you care about.
2. **Know the reference levels:** a baseline, and human-level or best-known performance if you have one. Then:
3. **avoidable bias** ~= training error - reference error,
4. **variance** ~= validation error - training error.
5. **Act on the bigger gap:**
6. High bias: a bigger or more flexible model, better features, less regularisation, training longer.
7. High variance: more data, regularisation, a simpler model, data augmentation.
8. A dev/test mismatch: get data that matches.
9. **Learning curves:** plot train and validation error against training-set size.
10. Converging curves with a high plateau mean bias, and more data won't help.
11. A large gap that is still closing means variance, and more data will help.
12. **Error analysis:** look at 100 misclassified dev examples and tally them by category or slice. Fix the categories that hold the most errors. A slice with a high error rate but few rows may matter less than a large slice with a moderate rate, unless it matters for fairness or the business.

Block 2: Worked example (45 min)

`code/week-08/day55_ml_strategy.py` on adult (learning curves with 5-fold CV). Verified error rates (majority-class reference: 0.2393):

model	n=781 train/val	n=11,721	n=39,073 train/val (gap)
logreg, 6 numeric columns	0.1915 / 0.1868	0.1871 / 0.1852	0.1851 / 0.1852 (0.0001)
unpruned tree	0.0000 / 0.2113	0.0000 / 0.1927	0.0001 / 0.1892 (0.1891)
HistGB	0.0000 / 0.1706	0.1071 / 0.1322	0.1158 / 0.1265 (0.0107)

Diagnosis. - **Logreg on numerics only:** train ~= val ~= 0.185 at every size. **High bias.** More data is useless, as the curve has been flat since n=4k. Act by adding the categorical features or a flexible model. - **Unpruned tree:** train 0 and val 0.19 with a 0.19 gap. **High variance.** More data helps slowly (0.211 -> 0.189). Act by regularising (depth, min leaf) or averaging (a forest). - **HistGB:** the gap shrinks from 0.17 to 0.011 as data grows, and val keeps falling. It is close to balanced. Further gains need better features or more data, not a different algorithm.

Error analysis (HistGB, out-of-fold predictions on train): - By **marital status**: Married-civ-spouse has an error rate of 0.2304 and **82.5% of all errors**. Never-married has 0.0292. The hard cases are married people, whose positive rate (0.4462) is closest to 50/50. - By **sex**: men have 0.1601 error and 83.7% of errors. Women have 0.0632. That is largely because women's positive rate

in this 1994 data is 0.1089, so most are easy negatives. - By **education**: HS-grad holds the most errors (29.7%) despite a low error rate (0.1177), because it is the largest group.

What to do next, concretely: features that separate high and low earners *among married people* (occupation x hours, capital gains). Day 58 builds on this.

Block 3: Coding lab (2h)

1. Reproduce the learning curves with `sklearn.model_selection.learning_curve`, and plot all three.
2. For the tree, add `max_depth` in {4, 8, 12} curves. Which one closes the gap best?
3. Do manual error analysis: print 30 misclassified married rows and invent 3 categories (e.g. "high hours but low-paying occupation"). Tally them.
4. Write a one-page "what to try next" memo with a table of options, expected effect, cost, and the evidence from your curves.
5. Stretch: plot *validation* error against the number of features added in Day 42's order. Is there a plateau?

Block 4: Practice problems (1h)

1. Human error 2%, train 8%, dev 10%. Bias or variance? *(Avoidable bias 6% > variance 2%. Work on bias.)*
2. Train 1%, dev 15%. Two actions? *(More data; regularisation or a simpler model.)*
3. Learning curves converge at 18% error and are flat. Is more data worth buying? *(No.)*
4. Dev from US users, production in India. What risk? *(Dev/test mismatch: build a dev set from the target distribution.)*
5. Slice A: 5% of rows, 40% error. Slice B: 60% of rows, 12% error. Which one has more errors in total? *(B: 7.2% vs 2%.)*

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Define avoidable bias and variance.
- Describe high-bias and high-variance learning curves.
- What did logreg on numerics show?
- Which slice held most errors?
- What is the purpose of a single-number metric?

Going deeper (optional)

Look up "ceiling analysis" for multi-stage pipelines: replace one stage at a time with perfect (ground-truth) output, and see how much the final metric improves. The stage with the biggest ceiling deserves your effort. You will use this in Month 4 for retrieval-augmented systems (is the retriever or the generator the bottleneck?).

Common mistakes / how to debug

- Tuning hyperparameters for weeks on a high-bias model.
- Error analysis on the test set (use dev or out-of-fold).
- Ignoring small slices that matter for fairness.

What to do next

Tomorrow: a systems project. Write k-means' hot loop in Rust, and measure whether it was worth it.

Day 56: PROJECT W8: Rust/PyO3 k-means kernel vs NumPy; Quiz 8

Review today: Day 55 (+1), Day 53 (+3), Day 49 (+7), Day 35 (+21; quick 15-min skim of its self-check).

Block 1: Concept lesson: when (and how) to drop to a compiled language (1h15)

Intuition. Python is slow per operation, and NumPy is fast because it hands whole arrays to compiled loops. Ask three questions before writing Rust: 1. Is this code actually the bottleneck? Profile it. 2. Can a better NumPy formulation fix it? Often yes. 3. Is the remaining cost in loops that NumPy can't express well (data-dependent branches, per-row reductions, parallelism)?

k-means' **assignment step** (find the nearest of k centroids for each of m points) is a good candidate. It is O(m.k.n), embarrassingly parallel over rows, and the naive NumPy version allocates large temporaries.

The NumPy tricks. - **Per-centroid loop:** k passes over X. - **Matmul trick:** argmin? ||x - c?||^2 = argmin? (||c?||^2 - 2x.c?), because ||x||^2 is the same for every j. One BLAS matrix multiply does the heavy lifting.

PyO3 + maturin (you used them on Day 29): annotate Rust functions with `#[pyfunction]`, accept NumPy arrays through the `numpy` crate as read-only views (no copy), and release the GIL. **rayon** turns `iter()` into `par_iter()` for multi-core parallelism. `maturin develop --release` builds and installs the module into your venv. Always use `--release`, since debug builds are many times slower.

Benchmark hygiene. - Use the same input and the same initial centroids. - Report best-of-N wall times. - Check that the outputs are *identical*. - Report the machine (here: 8 CPU threads). - Compare with the strongest baseline, not only the weakest.

Block 2: Worked example: reference results (45 min)

Build first: `source .venv/bin/activate && cd code/week-08/rust_kmeans && maturin develop --release`. The crate is `code/week-08/rust_kmeans/` (`src/lib.rs`, about 60 lines: `assign`, `assign_par` and `lloyd`). Then run `code/week-08/day56_week_project.py`. If the module isn't built, the script still runs the NumPy parts and says so.

Data: 200,000 x 16 blobs, k = 32. Verified on this box:

assignment step (best of 3)	time	labels identical to loop
NumPy per-centroid loop	307.7 ms	1.000000
NumPy matmul trick	26.6 ms	1.000000
Rust serial	34.9 ms	1.000000
Rust + rayon (8 threads)	**8.3 ms**	1.000000

Full Lloyd, 100 iterations, same init: NumPy 5.25 s vs Rust 0.70 s. Labels are identical and the maximum centroid difference is 0.0. **scikit-learn `KMeans(algorithm="lloyd")` with the same init: 0.22 s**, with labels agreeing with ours on 99.996% of rows.

Honest reading. 1. The *algorithmic* NumPy fix (the matmul trick) gave 11.6x, more than serial Rust (which is slower than the matmul because BLAS is highly optimised). 2. Rust's win comes from **parallelism** (about 3.2x over the matmul trick). 3. In the full loop, NumPy's centroid update (a Python loop over k with boolean masks) dominates, which is why Rust wins 7.5x there. 4. **scikit-learn beats our Rust** by about 3x, because its Cython implementation does chunked, cache-friendly, multithreaded passes that fuse assignment and update. Measuring the strong baseline is the whole point.

Block 3: Coding lab: the project (2h)

Deliverable: `notes/week08_rust_kmeans.md` with a timing table and a discussion. 1. **Read `src/lib.rs` (15 min):** find where the GIL is released, where `into_par_iter` is used, and how the inputs are borrowed without copying. 2. **Build and verify (15 min):** run the script and confirm that your numbers have the same *shape* as ours. Absolute times will differ by machine. 3. **Improve NumPy (20 min):** vectorise the centroid update with `np.add.at` or `np.bincount` per dimension, and re-time the full Lloyd loop. 4. **Improve Rust (40 min):** fuse assignment and update in one parallel pass. Each thread accumulates partial sums and counts, and these are reduced at the end. Re-time it and verify that the labels are still identical. 5. **Scaling study (20 min):** time `assign_par` with `RAYON_NUM_THREADS` = 1, 2, 4 and 8, and plot the speed-up. Explain why it is sub-linear. 6. **Write-up (10 min):** a table, a plot, and a recommendation: would you ship Rust here, or just use scikit-learn?

Rubric (self-grade out of 20):

criterion	points
Outputs verified identical (labels and centroids) for every implementation	5
Fair benchmark: same init, best-of-N, machine stated	3
Strong baseline (scikit-learn) included and discussed	3
One measured improvement (NumPy or Rust) with before/after numbers	4
Thread-scaling plot with an explanation	3
Clear recommendation tied to the numbers	2

Stretch: implement k-means++ initialisation in Rust (sequential sampling with parallel distance updates).

Block 4: Practice: Quiz 8 (1h), closed book, answers below

Block 5: Review (1h)

- Do the spaced reviews listed at the top.
- **Self-check:**
- Why can ||x||^2 be dropped in the assignment?
- Why does `--release` matter?
- Where did Rust's speed-up come from?
- Why did scikit-learn still win?
- Name three benchmark-hygiene rules.

Going deeper (optional)

Memory bandwidth bounds this kernel. Each pass reads 200,000 x 16 x 8 bytes = 25.6 MB of points. At about 8 ms that is around 3 GB/s of useful reads, which is far below typical memory bandwidth. That suggests compute (k = 32 distance evaluations per point), not bandwidth, is the main cost here. Check it with the f32 experiment. Try `f32` instead of `f64` and see whether SIMD and the halved memory help. Month 4's quantisation is the same idea at a much larger scale.

Common mistakes / how to debug

- Benchmarking a debug build.
- Timing the first call (it includes import and warm-up).
- Different initial centroids between implementations.
- Non-contiguous arrays passed to Rust: use `np.ascontiguousarray`.

What to do next

Week 9: make experiments reproducible, then build and deliver the Month-2 capstone, and start neural networks.

Weekly quiz: Week 8

1. How do you get PCA components and their variances from the SVD?
2. What did the PCA reconstruction error equal in our run?
3. Name two things t-SNE plots do not preserve.
4. Why did the multivariate Gaussian beat independent Gaussians on Day 51?
5. Write the biased MF prediction formula.
6. What did unregularised MF's test RMSE do between epochs 10 and 30?
7. Why does `y.rolling(24).mean()` leak, and what is the fix?
8. Shuffled vs time-split MAE for calendar + weather features?
9. Two calibration methods, and when to use each.
10. LinearSVC and logreg had the same AUC. What did Platt scaling change?
11. Define avoidable bias and variance (Ng style).
12. In the Rust project, which implementation won the assignment step, and which won the full Lloyd loop?

Answers

1. Centre X, then Xc = USumV^T. The components are the rows of V^T, and the variances are sigma^2/(m - 1).
2. The sum of the dropped eigenvalues divided by 64 (e.g. 4.914 at k=10).
3. Between-cluster distances, cluster sizes (also axes and density).
4. The features are strongly correlated, and only the full covariance models that.
5. r? = mu + b? + b_i + p?.q_i.
6. It rose from 0.9524 to 1.0173 (overfitting).
7. The window includes y? itself. Fix: `y.shift(1).rolling(24).mean()`.
8. 26.5 (shuffled) vs 48.7 (time split).

9. Platt/sigmoid for small data and S-shaped distortion. Isotonic for more data and arbitrary monotone distortion.
10. Brier went from 0.1142 to 0.0777 (calibration). AUC was unchanged.
11. Avoidable bias = train error - reference error. Variance = dev error - train error.
12. Rust + rayon won the assignment step (8.3 ms). scikit-learn won the full loop (0.22 s vs Rust 0.70 s).